Séance 2 — De la donnée à l'objet

Construire le modèle FoodFast

Classes · `enum` · `record` · `BigDecimal` · héritage · interfaces · collections · `Optional`

Objectifs de la séance

À la fin, vous savez :

- **AA2** — modéliser un domaine avec des classes, des `record` et des `enum`, avec encapsulation et invariants
- **AA3** — utiliser l'héritage, les interfaces et le polymorphisme
- **AA4** — choisir la bonne collection (`List`, `Set`, `Map`) et expliquer `equals` / `hashCode`

> Fil rouge : on construit, pas à pas, le modèle de FoodFast.

📖 Le support détaillé reprend chaque slide avec du code complet et compilé

1. Classe et objet

Pourquoi des classes ?

```
// S1 : un menu = des tableaux parallèles
String[] names    = {"Margherita", "Burger"};
int[] pricesCents = {1250, 990};
String[] sizes    = {"MEDIUM", "LARGE"};

// Supprimer le plat n° 1 ? Il faut toucher 3 tableaux.
// En oublier un = données incohérentes, sans erreur.
```

> Une **classe** regroupe les données **et** les règles d'un concept métier.
> Un **objet** est un exemplaire de cette classe : *cette* Margherita à 12,50 €.

Champs, constructeur, `this`

```
class Dish {
    String name;          // champ : propre à chaque objet
    int priceInCents;     // comme en S1, pour l'instant

    Dish(String name, int priceInCents) {   // constructeur
        this.name = name;                    // this = l'objet créé
        this.priceInCents = priceInCents;
    }
}

Dish pizza  = new Dish("Margherita", 1250);
Dish burger = new Dish("Burger", 990);
IO.println(pizza.name);   // Margherita
```

📖 Support §1.2 — Champs, constructeur, this

🟢🔒 Encapsulation : un objet invalide ne doit pas exister

```java
// ❌ champs accessibles
class Dish {
    String name;
    int priceInCents;
    // ...
}

pizza.priceInCents = -500;
// compile, et personne
// ne s'en rend compte
```

```java
// ✅ private + vérification
class Dish {
    private final String name;
    private final int priceInCents;

    Dish(String name, int price) {
        if (price < 0) throw new
          IllegalArgumentException(
            "Prix négatif");
        // ... affectations
    }
}
```

🟢 Des méthodes métier, pas des setters

```
// ❌ setters générés
order.setDishCount(-3);
order.setConfirmed(false);
// n'importe qui « dé-confirme »
// une commande
```

```
// ✅ ce que le métier permet
class Order {
  private int dishCount;
  private boolean confirmed;

  void addDish(int quantity) {
    if (quantity <= 0) throw new
      IllegalArgumentException(
        "Quantité : " + quantity);
    dishCount += quantity;
  }
  void confirm() { confirmed = true; }
}
```

> Un getter si quelqu'un en a besoin. Un setter presque jamais.

📖 Support §1.4 — Méthodes métier plutôt que setters

`static` contre instance

```
class Order {
    static final int MAX_QUANTITY = 20;  // un seul, partagé
    private int dishCount;               // un par commande

    void addDish(int quantity) {         // agit sur this
        if (quantity > MAX_QUANTITY) { /* ... refuser */ }
        dishCount += quantity;
    }
}

Order a = new Order();
Order b = new Order();
a.addDish(3);              // a : 3 plats, b : toujours 0
IO.println(Order.MAX_QUANTITY);   // appelé sur la classe
```

📖 Support §1.5 — static contre instance

2. `enum` et `record`

`enum` : un ensemble fermé de valeurs

```
// ❌ String : tout passe
String status = "PENDNG";
// faute de frappe,
// découverte en prod
```

```
// ✅ enum : le compilateur
// connaît les valeurs
enum DishSize {
    SMALL, MEDIUM, LARGE
}

enum OrderStatus {
    PENDING, CONFIRMED,
    PREPARING, IN_DELIVERY,
    DELIVERED, CANCELLED
}
```

`OrderStatus.PENDNG` → **erreur de compilation**. `OrderStatus.values()` liste les valeurs.

📖 Support §2.1 — enum

`switch` sur un `enum`

```
static String label(OrderStatus status) {
    return switch (status) {
        case PENDING     -> "En attente";
        case CONFIRMED   -> "Confirmée";
        case PREPARING   -> "En préparation";
        case IN_DELIVERY -> "En livraison";
        case DELIVERED   -> "Livrée";
        case CANCELLED   -> "Annulée";
    };
}
```

> Pas de `default` : si l'on ajoute `REFUNDED` à l'enum, ce `switch` **ne compile plus**. C'est voulu.

`record` : une valeur en une ligne

```
record Dish(String name, int priceInCents, DishSize size) {
    Dish {                                      // constructeur compact
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Nom vide");
        if (priceInCents < 0)
            throw new IllegalArgumentException("Prix négatif");
    }
}

Dish pizza = new Dish("Margherita", 1250, DishSize.MEDIUM);
pizza.name();    // "Margherita" : accesseur, sans get
IO.println(pizza);  // Dish[name=Margherita, priceInCents=1250, ...]
```

Généré : champs `private final`, constructeur, accesseurs, `equals`, `hashCode`, `toString`.

📖 Support §2.4 — record · §2.5 — Constructeur compact

3. Types riches

🤔 Prédisez la sortie

```
double a = 0.1 + 0.2;
IO.println(a);
IO.println(a == 0.3);
```

Que s'affiche-t-il ?

Réponse : `0.30000000000000004` et `false`

- Un `double` est stocké en **binaire** : `0.1` n'y a pas de valeur exacte
- En S1, vous avez contourné le problème avec des **centimes en `int`**
- Mais : 10 % de remise sur 12,45 € ? Quel arrondi ? Et l'affichage ?
- `BigDecimal` : un nombre **décimal exact**, avec un arrondi **choisi**

> Un montant, c'est toujours un `BigDecimal`. Jamais un `double`.

📖 Support §3.1 — Le problème du double

`BigDecimal` : créer et calculer

```
BigDecimal price = new BigDecimal("12.50");   // ✅ depuis une String
BigDecimal wrong = new BigDecimal(0.1);       // ❌ hérite de l'erreur
// wrong = 0.1000000000000000055511151231257827...

BigDecimal total = price.multiply(BigDecimal.valueOf(2));  // 25.00

total.add(new BigDecimal("2.90"));            // ❌ résultat perdu
total = total.add(new BigDecimal("2.90"));    // ✅ 27.90
```

> `BigDecimal` est **immuable**, comme `String` : chaque opération renvoie un **nouvel** objet.

`equals` contre `compareTo`

```
BigDecimal a = new BigDecimal("12.5");
BigDecimal b = new BigDecimal("12.50");
a.equals(b);          // false : compare valeur ET échelle
a.compareTo(b) == 0;  // true  : compare la valeur seulement
```

D'où la version finale de `Dish` : on **normalise** le prix.

```
record Dish(String name, BigDecimal price, DishSize size) {
    Dish {
        // ... mêmes vérifications (nom, null, prix >= 0)
        price = price.setScale(2, RoundingMode.HALF_UP);
    }
}
```

📖 Support §3.3 — equals contre compareTo

🟢 `LocalDateTime` et `UUID`

```
LocalDateTime createdAt = LocalDateTime.now();
LocalDateTime deadline  = createdAt.plusMinutes(45);
boolean late = LocalDateTime.now().isAfter(deadline);

UUID uuid = UUID.randomUUID();      // 122 bits aléatoires
String orderId = "order-" + uuid;  // order-3f2b8c1e-...-4...
```

- Dates : `java.time`, immuable. Oubliez `Date` et `Calendar`
- `UUID` : identifiant unique, sans compteur central

> 🔒 Un identifiant impossible à deviner **n'est pas** un contrôle d'accès.

📖 Support §3.5 — LocalDateTime · §3.6 — UUID

4. Héritage

🟢 Pourquoi l'héritage ?

Un client et un livreur ont tous les deux un `id`, un `name`, un `email`.

```
            Object          ← racine de toutes les classes
              │
        User (abstract)     ← id, name, email, role()
          ┌───┴────┐
      Customer    Courier   ← + address  /  rien de plus
```

> `Customer` **est un** `User`. On écrit le commun une fois, dans `User`.

La classe mère abstraite `User`

```
abstract class User {
    private final UUID id = UUID.randomUUID();
    private final String name;
    private final String email;

    protected User(String name, String email) {
        this.name = Objects.requireNonNull(name);
        this.email = Objects.requireNonNull(email);
    }
    public String getName() { return name; }
    // ... getId(), getEmail()
    public abstract String role();   // pas de corps
}
```

📖 Support §4.2 — extends, super, protected

`extends`, `super(...)`, `@Override`

```
class Customer extends User {
    private final String address;

    Customer(String name, String email, String address) {
        super(name, email);       // construit d'abord la partie User
        this.address = Objects.requireNonNull(address);
    }
    @Override public String role() { return "CLIENT"; }
}

User u = new Customer("Alice", "alice@example.com", "10 rue Haute");
u.getName();   // "Alice"  : hérité de User
u.role();      // "CLIENT" : la version de Customer
```

📖 Support §4.3 — @Override et abstract · §4.4 — Object

🟢 Composition plutôt qu'héritage

```
// ❌ « une commande EST
//    un client » ?
class Order extends Customer {
    // hérite de address,
    // email, role()...
}
```

```
// ✅ une commande A un client
class Order {
    private final Customer
        customer;
    // ...
}
```

- `extends` seulement si « **est un** » est vrai, et le reste
- Sinon : un champ (« **a un** »). C'est la composition
- Java : **une seule** classe mère par classe

5. Interfaces et polymorphisme

🟢 L'interface `Discount`

```
// ❌ avant : un if/else par type de remise, modifié à chaque ajout
// if (type.equals("PERCENT")) ... else if (type.equals("FIXED")) ...

interface Discount {
    BigDecimal apply(BigDecimal total);   // contrat : quoi, pas comment
}

record PercentageDiscount(int percent) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.multiply(BigDecimal.valueOf(100 - percent))
                    .divide(BigDecimal.valueOf(100), 2,
                            RoundingMode.HALF_UP);
    }
}
```

📖 Support §5.1 — Le problème · §5.2 — Interface et implémentations

🟢 Polymorphisme

```
record FixedDiscount(BigDecimal amount) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.subtract(amount).max(BigDecimal.ZERO);
    }
}

Discount[] promos = { new PercentageDiscount(10),
                      new FixedDiscount(new BigDecimal("5.00")) };
BigDecimal total = new BigDecimal("42.00");
for (Discount d : promos) {
    IO.println(d.apply(total));   // 37.80 puis 37.00
}
```

Nouvelle remise = nouvelle classe. La boucle **ne change pas**.

🔵 Interface ou classe abstraite ?

	Interface	Classe abstraite
Représente	une **capacité** (« sait appliquer »)	une **famille** (« est un utilisateur »)
Champs d'instance	non	oui
Constructeur	non	oui
Combien par classe	plusieurs	**une seule**
FoodFast	`Discount`	`User`

> Par défaut : une interface. Une classe abstraite si l'on partage de l'**état**.

📖 Support §5.4 — Interface ou classe abstraite

6. Collections

`List`, `Set`, `Map` sont des interfaces

```
List<Dish> menu = new ArrayList<>();        // ordonnée, doublons OK
Set<String> cuisines = new HashSet<>();     // pas de doublon
Map<String, Order> orders = new HashMap<>();  // clé → valeur
```

- À gauche l'**interface** (le contrat), à droite l'**implémentation**
- `<Dish>` : le **type des éléments**, vérifié à la compilation
- `<>` : Java déduit le type. Pas de primitif : `List<Integer>`

> Comme `Discount d = new FixedDiscount(...)`. Vous connaissez déjà le principe.

📖 Support §6.1 — Interfaces · §6.2 — Génériques

`List` : le menu

```
List<Dish> menu = new ArrayList<>();
menu.add(pizza);
menu.add(burger);

menu.size();        // 2
menu.get(0);        // pizza
menu.getFirst();    // pizza   (Java 21)
menu.getLast();     // burger  (Java 21)

for (Dish d : menu) { IO.println(d.name()); }

List<DishSize> sizes = List.of(DishSize.SMALL, DishSize.LARGE);
sizes.add(DishSize.MEDIUM);   // ❌ UnsupportedOperationException
```

📖 Support §6.3 — List

`Map` : les plats d'une commande

```
Map<Dish, Integer> items = new HashMap<>();

items.merge(pizza, 2, Integer::sum);   // pizza → 2
items.merge(pizza, 1, Integer::sum);   // pizza → 3
items.getOrDefault(burger, 0);         // 0, pas null

BigDecimal total = BigDecimal.ZERO;
for (Map.Entry<Dish, Integer> e : items.entrySet()) {
    BigDecimal qty = BigDecimal.valueOf(e.getValue());
    total = total.add(e.getKey().price().multiply(qty));
}
```

`Integer::sum` : « additionne l'ancienne et la nouvelle quantité ». Syntaxe vue en S3.

📖 Support §6.5 — Map

🤔 Prédisez la sortie

```
class PlainDish {                  // une classe, PAS un record
    private final String name;
    PlainDish(String name) { this.name = name; }
}

Set<PlainDish> favorites = new HashSet<>();
favorites.add(new PlainDish("Margherita"));

IO.println(favorites.contains(new PlainDish("Margherita")));
```

`true` ou `false` ?

Réponse : `false`. Il faut `equals` et `hashCode`

```java
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof PlainDish other)) return false;
    return name.equals(other.name);
}
@Override public int hashCode() {
    return Objects.hash(name);   // mêmes champs que equals
}
```

- Hérités d'`Object` : égalité d'**identité** (même objet en mémoire)
- `HashSet` cherche d'abord par `hashCode`, puis confirme par `equals`
- Contrat : `a.equals(b)` ⇒ `a.hashCode() == b.hashCode()`

> Un `record` les génère pour vous, sur **tous** ses champs.

📖 Support §6.6 — Le contrat equals / hashCode / toString

🔒 `equals` incorrect = données perdues

```
// ❌ Dish sans setScale
items.merge(new Dish("Pizza",
    new BigDecimal("12.5"), M),
    1, Integer::sum);
items.merge(new Dish("Pizza",
    new BigDecimal("12.50"), M),
    1, Integer::sum);
// 2 lignes « Pizza » x1
```

```
// ✅ constructeur compact
record Dish(String name,
            BigDecimal price,
            DishSize size) {
  Dish {
    // ... vérifications
    price = price.setScale(
      2, RoundingMode.HALF_UP);
  }
}
// 1 ligne « Pizza » x2
```

> Ne modifiez **jamais** un champ utilisé par `hashCode` d'une clé déjà dans une `Map`.

📖 Support §6.7 — record et BigDecimal · §6.8 — Clés mutables

🔒 Copie défensive : `getItems()`

```
// ❌ on rend la Map interne
Map<Dish, Integer> getItems() {
    return items;
}

// attaque, après confirm() :
order.getItems().put(pizza, -5);
// quantité négative, total faux
```

```
// ✅ une vue non modifiable
Map<Dish, Integer> getItems() {
    return Collections
        .unmodifiableMap(items);
}

order.getItems().put(pizza, -5);
// UnsupportedOperationException
```

> `private` ne suffit pas si l'on **rend** l'objet interne modifiable.

📖 Support §6.9 — Copie défensive

7. `Optional`

`Optional` : « peut-être absent », dans la signature

```
class DeliveryPlatform {
    private final Map<String, Order> orders = new HashMap<>();

    public Optional<Order> findOrderById(String id) {
        return Optional.ofNullable(orders.get(id));
    }
}

Optional<Order> found = platform.findOrderById("order-42");
if (found.isEmpty()) { IO.println("Commande inconnue"); }

Order order = platform.findOrderById(id).orElseThrow();
```

📖 Support §7.1 — Pourquoi · §7.2 — Utiliser Optional

Les pièges d'`Optional`

```
// ❌ get() sans vérifier :
//    la NPE devient une autre
//    exception, rien de gagné
opt.get();

// ❌ en champ ou en paramètre
private Optional<String> phone;
void send(Optional<String> to)

// ❌ renvoyer null quand le
//    retour est un Optional
return null;
```

```
// ✅ dire ce qu'on veut
opt.orElseThrow();
opt.orElse(defaultOrder);
opt.isPresent();

// ✅ Optional : uniquement
//    en type de RETOUR

// ✅ absent = vide
return Optional.empty();
```

📖 Support §7.3 — Les pièges

Ce qu'il faut retenir

- **Classe** : champs `private`, invariants vérifiés dans le constructeur, méthodes métier
- `enum` pour les valeurs fermées, `record` pour les valeurs immuables
- **Montants** : `BigDecimal` depuis une `String`, comparés avec `compareTo`
- **Héritage** si « est un » ; sinon **composition**. **Interface** pour un contrat
- **Collections** : `List` / `Set` / `Map` déclarées par leur interface ; `equals` **et** `hashCode`
- `Optional` en retour d'une recherche ; 🔒 ne jamais exposer une collection interne

Pour aller plus loin

Dans le support détaillé :

- 🔵 Arrondis `HALF_UP` / `HALF_EVEN`, enum avec champs, méthodes `default`
- 🔵 `LinkedHashMap`, `TreeMap`, `ArrayDeque` et leurs complexités
- 🟣 Écrire ses propres génériques
- 🟣 `sealed interface Discount` et `switch` avec motifs
- 🟣 Valider avant `super(...)` (JEP 513)

📖 Support §8 — Le modèle complet · §10 — Ressources